CS3500 Operating Systems
Lab 7: File System

Release: Monday, 12 October 2026

Due: Sunday, 1 November 2026, 11:59 PM IST

1 Background Reading & Environment Setup

1.1 Introduction

Inspect an inode, print its block tree, extend the maximum file size, and implement symbolic links.

All tasks in Sections 2–5 are required. Section 7 is optional and ungraded.

1.2 Background Reading

Read the current online xv6 book's Chapter 10: File system and Chapter 11: Logging. Chapter numbers differ in older editions; use the titles to identify the corresponding material. Review these source files:

- `kernel/fs.h`, the on-disk layout: `struct superblock`, `struct dinode`, `struct dirent`
- `kernel/fs.c`, `balloc`, `bfree`, `bmap`, `itrunc`, `readi`, `writei`, `namei`
- `kernel/file.h`, the in-memory `struct inode`
- `kernel/file.c`, write chunking and final file close
- `kernel/bio.c`, the buffer cache: `bread`, `bwrite`, `brelse`
- `kernel/log.c`, the logging layer: `begin_op`, `end_op`, `log_write`
- `kernel/sysfile.c`, the system call layer for files
- `mkfs/mkfs.c`, the program that builds `fs.img`

Understand the inode layout and filesystem transactions. Log modified buffers with `log_write` and release every buffer obtained with `bread` using `brelse`.

1.3 Setting Up the Lab

Start Docker Desktop, then run these commands in your host terminal:

```
git clone --branch cs3500-filesystem-starter \
    https://github.com/skbgp/CS3500-File-System-Lab.git
cd CS3500-File-System-Lab
docker run --rm -it --tmpfs /tmp -v "$PWD":/home/xv6-labs \
    -w /home/xv6-labs nandhagk/xv6-tools:latest sh
```

Run the commands in order. Docker mounts the current directory (`"$PWD"`) at `/home/xv6-labs`, so enter the cloned repository first. In the container shell, run:

```
make qemu
```

Run `usertests -q` inside xv6 to check the starter. Tests for the new features will fail until you implement them. To exit QEMU, press Ctrl+A, release the keys, then press X.

Leave `CLONE_OFF` defined in `kernel/bref.h`; it is already set in the starter and disables reference counting for the optional cloning challenge.

After changing the inode layout, or if the disk image becomes inconsistent, exit QEMU, delete `fs.img` from the repository in the host or container shell, and rebuild with `make qemu`. This deletes files created inside xv6.

Testing while you work

Exit QEMU and run selected tests in the container shell while developing:

```
python3 grade-lab-fs --only tree_direct
python3 grade-lab-fs --only large_boundaries
python3 grade-lab-fs --only symlinks
```

Use `python3 grade-lab-fs -list` to list cases. Selected runs report marks only for those cases. Run the full `make grade` before submitting; it may take 15–25 minutes.

2 Inspect an on-disk inode

Run `fsinfo README` inside xv6. It prints the superblock and inode using the supplied `istat` syscall. Before changing the inode layout, expect:

```
$ fsinfo README
superblock: size 200000 nblocks 199328 ninodes 200 nlog 241
            logstart 2 inodestart 243 bmapstart 256 refstart 281

inode 2: type 2 nlink 1 size 2441
  addrs[0] = 673
  addrs[1] = 674
  addrs[2] = 675
  addrs[3] = 0
  addrs[4] = 0
  addrs[5] = 0
  addrs[6] = 0
  addrs[7] = 0
  addrs[8] = 0
  addrs[9] = 0
  addrs[10] = 0
  addrs[11] = 0
  indirect = 0
```

Write answers to the following in `answers-fs.txt`.

1. For each superblock field printed above, explain its value, how `mkfs` sets it, and where the kernel uses it, if at all. Also identify where the kernel reads the superblock at boot.

2. The `README` above occupies three data blocks for 2441 bytes. Explain the relationship between `size` and its allocated data blocks. Why does counting nonzero inode entries alone not give the data-block count for a larger file? What initializes unused bytes in a newly allocated final block, and can a file read return bytes beyond `size`?

3. Given only the inode above, which disk block holds byte offset 1500 of the file? Show the calculation using zero-based offsets. Explain why offset 300000 has no block in this file and cannot be represented by the starter layout. After Section 4, what file size and additional block-pointer information would be needed to resolve that offset?

4. In stock xv6, what is the largest file that can be created, in blocks and in bytes? Derive the number from the constants in `kernel/fs.h` rather than quoting it.

5. In the starter layout, `addrs[]` has `NDIRECT+1` entries, but only the first `NDIRECT` entries can directly name data blocks. Explain what the last entry points to, and why a file's data blocks are not, in general, contiguous on disk.

3 Print an inode's block tree

Write a function that prints a file's block tree. You will use it to check your large-file implementation.

The supplied `iprint(char *path)` system call looks up `path` and calls `itreeprint()` in `fs.c`. Implement `itreeprint(struct inode *ip)` to print the inode's blocks in the format below.

Run `print_itree path` inside xv6, replacing `path` with a file name; for example, `print_itree README`. The following is an abbreviated example for a 269-block file *after* Section 4. The omission marker is explanatory and must not appear in your program's output:

```
inode 24 type 2 size 275456
..0: 700
..1: 701
..2: 702
..3: 703
..4: 704
..5: 705
..6: 706
..7: 707
..8: 708
..9: 709
..10: 710
..indirect: 711
.. ..0: 712
.. ..1: 713
[entries 2 through 254 omitted here]
.. ..255: 967
..double-indirect: 968
.. ..0: 969
.. .. ..0: 970
.. .. ..1: 971
```

Print the inode number, type, and byte size on the first line. Then visit all nonzero pointers in increasing index order within each parent:

- Direct blocks and indirection roots: prefix `..`
- Children of either indirection root: prefix `.. ..`
- Data blocks beneath a doubly-indirect child: prefix `.. .. ..`

Use no leading space. Print each entry's index and block number; label the roots `indirect` and `double-indirect`. Skip zero pointers. Inode and disk block numbers may differ from the example.

Implement the starter's twelve direct pointers and single-indirect traversal first, then update the printer for Section 4. Printing must not allocate or modify blocks.

Hints

- Use `itrunc`'s traversal as a starting point.
- Release every buffer with `brelse`, including on error paths.

- The caller already holds the inode lock; do not lock it again. Keep inspection syscalls inside a transaction, since `iput` may reclaim a concurrently unlinked inode.

4 Large files

Keep the supplied settings in `kernel/param.h`: `FSSIZE=200000`, `MAXOPBLOCKS=80`, and `LOGBLOCKS=240`. They provide disk and log space for the required large-file operations.

The starter limits files to 268 data blocks, or $268 \times$ `BSIZE` bytes (`BSIZE` is 1024). An inode contains 12 "direct" block numbers and one "singly-indirect" block number, which refers to a block that holds up to 256 more block numbers, for a total of $12 + 256 = 268$ blocks.

Add a "doubly-indirect" pointer to each inode. It points to a block with 256 addresses of singly-indirect blocks; each of those can name 256 data blocks. With one direct slot repurposed, the limit becomes $11 + 256 + 256 \times 256 = 65{,}803$ data blocks.

Extend `bmap()` to handle this third level without changing the on-disk inode size. The first 11 entries, `ip->addrs[0]` through `ip->addrs[10]`, name data blocks. The entry `ip->addrs[11]` names the singly-indirect block, and `ip->addrs[12]` names the doubly-indirect block. These are zero-based C array indices.

A correct implementation must support 65,803 data blocks per file and leave `usertests -q` passing. Run the supplied `bigfile` program: it writes and reads back a maximum-size file, checks the size limit, and removes the file. Use `bigfiletest` to check the direct, indirect, and doubly-indirect boundaries on a smaller file.

Hints

- Sketch the pointer tree and derive the outer and inner indices from a logical block number. Use `itreeprint` to inspect it.
- Set `NDIRECT` to 11 and update `MAXFILE`. Keep 13 address slots in both `struct inode` and `struct dinode`; the new array length is `NDIRECT+2`. Preserve the on-disk inode size.
- Ensure `istat` copies all 13 slots and `fsinfo` displays both indirection roots. Check `mkfs`'s indexing after the layout change. It supports direct and single-indirect files; keep its bound check within `NDIRECT+NINDIRECT`. The supplied programs fit within this range.
- Delete and rebuild `fs.img` after changing `NDIRECT`.
- Allocate pointer blocks only when needed. Handle allocation failure without losing references to blocks that must later be reclaimed.
- Make sure `itrunc` frees all blocks of a file, including doubly-indirect blocks.

Answer the following in `answers-fs.txt`: a maximum-size file needs one singly-indirect block, one doubly-indirect block, and 256 singly-indirect blocks below it. What fraction of the file's allocated blocks hold pointers rather than data? Count its 65,803 data blocks and 258 pointer blocks; exclude the inode-table block, directory-entry block, filesystem log, and buffer-cache memory. How would that fraction change for a maximum-size file if `BSIZE` were 4096, while block addresses remained 4 bytes and the inode retained 11 direct slots and two indirection roots? Treat this as a theoretical address-capacity calculation with a

sufficiently large disk and sufficiently wide size/offset integers; do not change `BSIZE` in your implementation.

5 Symbolic links

A symbolic link has its own inode and stores a pathname. Opening it normally looks up that pathname. A hard link is different: it is another directory entry for an existing inode. This part of the lab implements symbolic-link creation and final-component symlink following in `open()`.

Other kernel pathname operations retain their existing behavior; in particular, `link()` and `unlink()` operate on the final symlink inode itself. Do not change `namei()` to follow links globally. The existing user-level `stat()` helper follows links indirectly because it calls `open()`. The supplied `istat` and `iprint` inspection syscalls continue to inspect the final inode named by the path without following it.

You will implement the `symlink(char *target, char *path)` system call, which creates a new symbolic link at `path` that stores `target`. Return 0 on success and -1 on failure. The destination must not exist, including as a dangling symlink; the target need not exist. Reject empty paths and strings that do not fit in `MAXPATH` bytes including the terminating zero. Failed creation must not leave a directory entry or leak an inode or block.

For this lab, follow symlinks only in the final pathname component. Resolve absolute targets from the root. Resolve relative targets from the directory containing the symlink entry used for this lookup, regardless of the calling process's current directory. For example, if `dir/link` stores `file`, opening `dir/link` opens `dir/file`; if it stores `../file`, the lookup uses `dir/../file`. Apply this rule again at each link in a chain, using that link's own containing directory. If the same symlink inode has hard links in different directories, use the directory of the entry being followed. Reject an expanded pathname that cannot fit in `MAXPATH` bytes. Intermediate components must be actual directories. Links to directories are outside the required scope.

Store the target as a zero-terminated string, including the terminator in the inode size. For example, `target` occupies seven bytes.

Inside xv6, run `symlinktest` and `nofollowtest`. For failed-creation cleanup, exit QEMU and run `python3 grade-lab-fs -only symlink_failure` in the container shell; the grader prepares a nearly full disk. Use `symlink target path` for manual tests.

Hints

- Add the syscall number and dispatcher entry in `kernel/syscall.h` and `kernel/syscall.c`, the declaration in `user/user.h`, and the stub entry in `user/usys.pl`. Implement `sys_symlink` in `kernel/sysfile.c`. Reuse existing declarations if provided.
- Add a new file type (`T_SYMLINK`) to `kernel/stat.h` to represent a symbolic link.
- Add `O_NOFOLLOW` in `kernel/fcntl.h`, using a bit that does not overlap existing flags.
- Check `writei`'s return value; a short write is a failed creation.
- `create()` publishes the directory entry before returning. Either undo a failed creation or initialize the link before publishing its entry. Reclaim the new inode and blocks; preserve existing files. `end_op()` does not roll back a failed syscall.

- Before following a link, validate its stored contents. A valid target has an inode size between 2 and `MAXPATH` bytes inclusive. Reading the target must return exactly that size, and its first zero byte must be the final stored byte. Otherwise, following the link fails with -1.
- In `open`, following a missing target must fail. With `O_NOFOLLOW`, open the link inode using the requested access mode; reads return the target including its terminator. Validate its contents whenever it is subsequently followed. Apply `O_TRUNC` only to a resolved regular file. `O_CREATE` on an existing symlink may fail; creating its missing target is not required.
- Follow links iteratively: ten links ending in a regular file must succeed; attempting an eleventh must return -1. This also stops cycles. Track each entry's pathname for relative targets. Release the current inode lock before the next lookup and balance all references.

Answer the following in `answers-fs.txt`:

1. If you hold a symlink inode lock while looking up its target, what can go wrong, and what does your implementation do instead?
2. Explain how your implementation resolves the relative targets in a chain such as `a/first` storing `../b/second`, where `b/second` stores `file`. Which directory is used at each step?

6 Submission

6.1 Time Spent

Create a new file, `time.txt`, and put in it a single integer, the number of hours you spent on the lab, rounding up any partial hour.

6.2 Answers

Write your answers to questions in Sections 2, 4, and 5 in `answers-fs.txt`, labelled by section and question number.

6.3 References

Create `reference.txt` and list all sources you consulted, including links where available. For each student collaborator, give their name and roll number, and describe the specific parts on which you collaborated. For other collaborators, give their name and role. All collaboration must follow the course collaboration policy.

If you used generative AI to understand concepts, debug code, or generate any part of the solution, name the AI tool and model (if known), state **which parts** it assisted with, and include the corresponding prompt logs as tracked files. Identify them in `reference.txt`. If you used no external sources, collaboration, or AI, state that explicitly.

6.4 Submit

Submit one ZIP file on Moodle, named with your roll number, for example `CS24B046.zip`. Complete the following steps from the repository root. Run Git and archive commands in

your host terminal; the Docker image does not include Git.

1. Exit QEMU and run `make grade` in the container shell. Review the results and fix any failures before submitting.

2. Use `git status` to review your changes. Add your source changes, any added tests, `answers-fs.txt`, `time.txt`, `reference.txt`, and prompt logs if applicable, then commit them. Do not add binaries or `fs.img`.

3. Exit the container and run `make zipball` in your host terminal to create `lab.zip`. Only committed files are included, so commit all required changes before creating the archive.

4. Check the ZIP contains your final source and required text files. Rename `lab.zip` to `<ROLL_NUMBER>.zip`, for example `CS24B046.zip`, and upload it to Moodle before the deadline.

Additional grading tests may check any required behavior in this handout.

7 Optional Challenge Exercises

These exercises are optional and ungraded. They do not replace required tasks.

- On a separate branch, implement copy-on-write file cloning with `clone(src,dst)`. Use `kernel/bref.c` for reference counts, unshare blocks before writes, and reclaim shared trees safely.
- Support triple-indirect blocks. State which inode slot you replace, preserve the inode size, and account for file-size and offset integer widths as well as pointer capacity when calculating the new limit.
- Support symbolic links to directories, with pathname resolution through intermediate symlink components.
- Add a `df` command reporting reserved, allocated, and free blocks.

Acknowledgment

The large-file and symbolic-link exercises are adapted from the MIT 6.1810/6.828 Operating Systems Engineering filesystem lab:
https://pdos.csail.mit.edu/6.828/2025/labs/fs.html
Inode inspection, tree printing, and relative-target requirements are course extensions.